National University of Computer and Emerging Sciences



Laboratory Manuals

for

Computer Networks - Lab

(CL -3001)

Course Instructor	Dr. Abdul Qadeer
Lab Instructor	Ms. Halima Sadia
Section	BCS-5A
Semester	Fall 2026

Department of Computer Science
FAST-NU, Lahore, Pakistan

Lab Manual 04

CLOs

- CLO 2: Demonstrate an understanding of the fundamental concepts of socket programming, types of protocols, and IP addressing.

Objectives

By the end of this lab, students should be able to:

- Implement a basic UDP client-server application using Python sockets.
- Understand how UDP requests and responses are exchanged between a client and server.
- Use UDP socket functions such as sendto() and recvfrom().
- Implement basic request processing and maintain records using a Python list.

Prerequisites

Students should have basic knowledge of:

- Python functions, strings, lists, and conditional statements.
- Client-server communication.
- UDP and transport-layer protocols.
- Basic IP addressing.

UDP (User Datagram Protocol)

UDP is a connectionless transport-layer protocol that allows applications to send data without establishing a connection before transmission.

UDP does not guarantee delivery, ordering, or duplicate protection. It has lower communication overhead than connection-oriented protocols and is useful for applications where low latency and simple communication are important.

Characteristics of UDP

- UDP does not guarantee delivery or ordering of packets.
- UDP does not require a connection setup before sending data.
- UDP has lower communication overhead.
- UDP communication uses datagrams.
- UDP uses source and destination port numbers to identify applications.

In this lab, UDP is used to exchange library requests and responses between a client and server without establishing a connection.

Lab Statement: Library Borrowing and Return System

Total Marks: 20

Design and implement a UDP-based Library Borrowing and Return System using the provided UDP_Client.py and UDP_Server.py files.

The client will send a request to the server. The server will receive the request, process it, update the library records, and send an appropriate response back to the client.

The server must remain running and process multiple consecutive requests without being restarted.

1. Client-Server Workflow

For each request, the application should follow this process:

1. Start the UDP server.
2. Start the UDP client in a separate terminal.
3. The client sends a request to the server using sendto().
4. The server receives the request using recvfrom().
5. The server processes the request.
6. The server updates the library records if required.
7. The server sends a response to the client using sendto().
8. The client receives and displays the response using recvfrom().
9. The server displays the current list of students.
10. The server remains running and waits for the next request.

2. Library Records

For this lab, the library database should be represented using a Python list containing the roll numbers of students who currently have a borrowed book.

Example:

```
students = ["24-4159", "24-4160", "24-4172"]
```

When a student successfully borrows a book, their roll number is added to the list.

When a student successfully returns a book, their roll number is removed from the list.

The remaining students must retain their original order after a student is removed.

3. Client Request Format

The client must send one of the following request formats:

Borrow

```
YY-AAAA-BR
```

Example:

```
24-4159-BR
```

Return

```
YY-AAAA-RT
```

Example:

```
24-4159-RT
```

Here, YY-AAAA represents the student's roll number.

The client should send only the request string, such as:

```
24-4159-BR
```

Note: The client must send the request in one of the specified formats: YY-AAAA-BR for borrowing or YY-AAAA-RT for returning. If the request does not follow either format, the server must respond with **Invalid request format.** and must not modify the library records.

4. Borrow Book

When the server receives a Borrow request:

Case 1: Student has not borrowed a book

If the student's roll number is not present in the library records:

- Add the student's roll number to the list.
- Send the following response to the client:

```
Book issued to Student YY-AAAA.
```

Example:

```
Book issued to Student 24-4159.
```

Case 2: Student has already borrowed a book

If the student's roll number is already present in the library records:

- Do not add the student again.
- Send the following response:

```
You have already borrowed a book.
```

5. Return Book

When the server receives a Return request:

Case 1: Student has borrowed a book

If the student's roll number is present in the library records:

- Remove the student's roll number from the list.
- Send the following response:

```
Book returned by Student YY-AAAA. Thank you.
```

Example:

```
Book returned by Student 24-4159. Thank you.
```

The remaining students must retain their original order.

Case 2: Student has not borrowed a book

If the student's roll number is not present in the library records:

- Do not modify the list.
- Send the following response:

```
You have not borrowed a book. Contact Library Administrator.
```

6. Multiple Requests

The server must remain running after processing each request.

The server must be able to process multiple consecutive requests without being restarted.

For example:

```
Borrow → Borrow → Duplicate Borrow → Return → Return Without Borrowing
```

The server should process each request and send a response to the client before waiting for the next request.

7. Display Current Library Records

After processing every borrow or return request, the server must display the current list of students.

For example:

```
Current students: ['24-4159', '24-4160']
```

If all books have been returned:

```
Current students: []
```

8. Required Test Cases

You must test the system using the following cases.

Test Case	Request	Expected Result
Successful Borrow	24-4159-BR	Student is added to the list
Duplicate Borrow	24-4159-BR again	Student is not added again
Successful Return	24-4159-RT	Student is removed from the list
Return Without Borrowing	24-4159-RT again	Request is rejected
Multiple Requests	Several requests consecutively	Server remains running and processes all requests

Use an appropriate student roll number from your section for testing.

9. Example Execution

First Request — Borrow

Client:

```
Enter request: 24-4159-BR
```

Server:

```
Request received: 24-4159-BR
Book issued to Student 24-4159.
Current students: ['24-4159']
```

Client:

```
Book issued to Student 24-4159.
```

Second Request — Duplicate Borrow

Client:

```
Enter request: 24-4159-BR
```

Server:

```
Request received: 24-4159-BR
You have already borrowed a book.
Current students: ['24-4159']
```

Third Request — Return

Client:

```
Enter request: 24-4159-RT
```

Server:

```
Request received: 24-4159-RT
Book returned by Student 24-4159. Thank you.
Current students: []
```

10. Submission Requirements

- Submit the following:
- Modified UDP_Client.py
- Modified UDP_Server.py
- Screenshots showing:
    1. Successful book borrowing
    2. Duplicate borrowing
    3. Successful book return
    4. Return without borrowing
    5. Server processing multiple requests without restarting

11. Marks Distribution

Component	Marks
UDP client implementation	4
UDP server implementation	5

Borrow operation	3
Return operation	3
Duplicate/unsuccessful operation handling	2
Multiple requests and database display	2
Required testing and screenshots	1
Total	**20**

Important Notes

- Use the provided UDP_Client.py and UDP_Server.py as the starting files.
- The server must be started before the client.
- The server must remain running while processing multiple requests.
- The library records should be maintained using a Python list.
- Do not add duplicate student roll numbers.
- Do not remove a student who is not currently in the list.
- The server must send a response to the client for every correctly formatted borrow or return request.